

SUNPOR — Operator Forms & AI Logic

Reply regarding the original operator forms and future AI predictions

Thank you for the detailed review and for redesigning the forms. Your goal is exactly right: we need structured operator knowledge that we can later correlate with process data and quality results.

Below is the logic behind the **original forms**, how they connect to AI predictions, and how we see the full workflow. We also comment on how your redesigned categories fit with that approach.

1. Reasoning behind the original forms

The original forms were built from three sources:

- the Excel input mask (*Input_Mask_Extrusion*) from Client
- the questionnaire / plant process description
- the idea that operators must enter data **quickly during a shift**

So the design was intentionally **lean and event-based**, not a full incident investigation form. Each form answers one clear question:

Form	What it captures	Why it exists
Production start	Line, material/recipe, trial, shift, start time, comment	Creates the production run container that everything else is linked to
Extruder events	Phase / maintenance events (heating, shutdown, screen, nozzle, low production) + time + reason	Labels process phases and maintenance actions for Process State / Nozzle–Screen learning
Granulator events	Knife change / grinding + time	Labels for knife wear / grain distribution models
Cleaning	Water bath / centrifuge / cleaning work + reason	Labels cleaning phases so they are not treated as faults
Faults	Mechanical / electrical + subtype + time	Labels real disturbances for Process State and fault patterns
Material behavior	Observation type (lumps, twin beads, pentane, foaming...) + severity + time	Operator “floor quality” labels for Material Behavior Risk
Material blocking	Reason + time range	Later ground truth for bad batches (Predictive Quality)
Daily quality	Open holes %, sieve %, foaming OK/NOK + shift	Regular quality ground truth for Predictive Quality

Core idea of the original design:

- Every entry is attached to a **production run**.
- Dropdowns force **structured labels** (not free text only).
- We keep the form short so operators actually fill it every shift.
- AI then joins: **WinCC signals + operator labels + later quality** for the same run.

So the original forms were designed as **labels and context**, not as a complete root-cause investigation record.

2. Which AI predictions we expect from this data

From the collected forms + live signals, we expect these predictions (aligned with our requirements document):

- **Process State** — current phase (heating, startup, stable, low production, cleaning, empty run, cooling, shutdown, fault)
- **Low-production cause & severity** — when throughput is low: planned vs feeder vs material vs process
- **Early anomaly detection** — early doser/process drift before SCADA alarms
- **Material behavior risk** — rising risk of lumps, twin beads, pentane/foaming issues
- **Nozzle / screen / pressure risk** — clogging / screen change need
- **Granulator / knife wear risk** — knife condition / grain-distribution risk
- **Predictive quality** — quality risk during production (open holes, sieve, foaming, blocking), days before lab confirmation

- **Maintenance prioritization** — ranked maintenance hints

3. Relationships / correlations the model should learn

Exactly the chain you described:

- What did the operator observe? → Material Behavior / Faults / comments
- What happened before? → Extruder/Granulator/Cleaning events + signal trends before the event
- Why did they intervene? → Level 2/3 + reason (e.g. screen change because high ΔP / planned)
- What action was taken? → event type (knife change, nozzle grinding, cleaning, etc.)
- What was the result? → process recovery in signals + later Daily Quality / Material Blocking
- What was the later quality outcome? → Daily Quality + Material Blocking + lab/QA decision

Examples of concrete correlations:

- Rising screen ΔP + “Screen change / High differential pressure” → earlier screen-risk warning next time
- Knife torque drift + “Knife grinding” → knife-wear prediction
- Material behavior “twin beads” + process-water/granulator signals → material-risk score
- Stable-run signal pattern + later foaming NOK → predictive quality during production

4. Links between operator inputs and future predictions

Operator input	Used as	Feeds prediction
Production start (material, trial, shift)	Context for every model	All capabilities
Extruder Level 2/3 + time	Phase/maintenance labels	Process State, Low Production, Nozzle/Screen
Granulator knife events	Maintenance labels	Granulator / Knife Wear
Cleaning events	Planned non-production labels	Process State (avoid false alarms)
Faults	Disturbance labels	Process State, anomaly validation
Material behavior + severity	Observation labels	Material Behavior Risk, Predictive Quality
Material blocking	Bad-batch ground truth	Predictive Quality
Daily quality	Quality ground truth	Predictive Quality, Granulator, Nozzle/Screen

Without these labels, we only have sensor curves. With them, we can say: **“This signal pattern + this operator observation → later this quality/maintenance outcome.”**

5. Complete prediction workflow (data → AI output)

1. **Production Start** opens a run (line, material, shift, recipe/order if available).
2. During the run, WinCC signals are ingested continuously (features over 1/5/15/30 min windows).
3. Operators log events/observations on the forms (structured dropdowns + time + comment).
4. AI computes live outputs (process state, anomalies, risks) and stores them with the run + input window.
5. Later, Daily Quality / Material Blocking / QA results are attached to the same run.
6. Correlation layer builds labeled examples: *(signals + events + observations during run) → (later quality / maintenance outcome)*.
7. Over time, rule-based risks improve into ML predictions for new runs **during production**.

6. How your redesigned list fits

Your data categories are very good. Many already exist in our model (run, line, material, event time, event type, process area via Level 1/2, comments, quality results).

Some of your fields are **richer than the original lean forms**, especially:

- time when problem first occurred / noticed / resolved
- duration until detection / resolution

- process conditions observed
- actions taken
- suspected cause
- events immediately before
- people notified
- result of intervention
- linked quality inspection / final QA decision

We agree these strengthen AI learning. The only risk is operator load: if every event requires too many fields, data quality drops.

Proposal for the next step:

1. Keep the original **form separation by purpose** (start, events, material behavior, quality, blocking) — easy for daily use.
2. Enrich the forms with the most valuable fields from your redesign, especially for incidents/faults: noticed time / resolved time, suspected cause, action taken, result of intervention, link to quality inspection / final decision.
3. Keep optional/advanced fields for serious events, not for every routine entry (e.g. planned screen change).

This way we collect the right AI information from the beginning, without making normal shift logging too heavy.

We are happy to go through your redesigned forms field-by-field and mark: **must-have for AI from day one**, **useful later**, and **optional / nice-to-have**. Then we can freeze one shared form design before implementation continues.